Artist Platform — Team Onboarding & Technical Blueprint

Document status: Prototype / Pre-MVP
Purpose: Give a new team member a complete understanding of the product, architecture, current scope, and development direction.

---

1. What Are We Building?

We are building a platform dedicated to **artists and the people who discover, support, buy from, or hire them**.

The platform combines three major ideas:

1. **Artist social/community platform**
2. **Artist marketplace and booking platform**
3. **Trust and transaction-protection system**

The long-term vision is:

> **Discover artists. Support artists. Buy from artists. Book artists. Build trusted relationships.**

We do not want to simply create another Instagram clone.

The major differentiator is **trust + artist commerce**.

---

2. What Problem Are We Solving?

Artists currently need multiple platforms.

For example:

- Instagram → audience and discovery
- Spotify → music
- Etsy → products
- Fiverr → services
- WhatsApp → negotiations
- Separate payment methods → transactions

This creates fragmented experiences.

Our platform aims to bring the important parts together:

```
Artist
  ↓
Verification
  ↓
Profile / Portfolio
  ↓
Community
  ↓
Products / Services / Bookings
  ↓
Protected Deal
  ↓
Completion
```

---

3. Who Uses the Platform?

A. Normal User

A user can:

- Create an account
- Discover artists
- Search artists
- Follow artists
- Appreciate content
- Comment / participate in discussions
- View artist portfolios
- Buy products
- Book artists
- Chat with artists
- Track orders/bookings
- Report problems
- Receive notifications

---

4. Artist Account

Artists have a separate onboarding process.

An artist cannot simply declare themselves verified.

They apply to become an artist.

Artist Application

The artist can provide:

- Name
- Profile information
- Artist category
- Bio
- Social-media profile
- Portfolio/work URL
- Relevant work samples
- Other verification information

The application enters:

```
PENDING
```

---

5. Artist Verification System

One of the important ideas is proving that the person actually controls the social account they submitted.

Example:

Artist submits:

```
instagram.com/exampleartist
```

Our platform generates a unique verification code:

```
AV-82X91
```

The artist puts that code somewhere on their controlled social profile, such as a temporary post/story/bio depending on the verification method we implement.

Our admin team checks it.

Result

```
Verified
```

or

```
Rejected
```

or

```
More Information Required
```

This helps reduce fake artists and impersonation.

---

6. Artist Profile

A verified artist receives a dedicated profile.

Possible sections:

- Profile photo
- Artist name
- Verification status
- Bio
- Category
- Portfolio
- Posts
- Products
- Services
- Booking information
- Social links
- Supporters/followers
- Reviews/reputation
- Availability

The exact profile structure will be finalized during UX research.

---

7. The Social Layer

The platform should feel like a genuine artist community.

Users can discover:

- Artwork
- Music
- Performances
- Photography
- Videos
- Behind-the-scenes content
- Announcements
- New releases
- Events

---

8. Platform Terminology

We do not necessarily want to copy social-media terminology.

We are exploring more artist-specific language.

Possible alternatives to "Like"

- Appreciate
- Applause
- Encore
- Ovation

Our strongest current candidate is:

> **Applause**

because it works for many artist categories.

Possible alternatives to "Followers"

- Supporters
- Audience
- Community

These are still under research.

Important

Terminology is **not final yet**.

The team should treat these as product experiments, not hard requirements.

---

9. Marketplace

Artists will eventually be able to sell things.

Examples:

- Paintings
- Prints
- Sculptures
- Crafts
- Merchandise
- Digital artwork
- Custom commissions

A user can discover an item and start a protected transaction.

---

10. Booking System

Artists can also offer services.

Examples:

- Singer for an event
- Band performance
- DJ booking
- Photographer
- Dancer
- Painter
- Workshop
- Custom artwork
- Commission work

A user can request/book an artist directly through the platform.

---

11. Protected Transaction Concept

This is one of the most important long-term features.

The original concept is an **escrow-like transaction-protection system**.

The long-term concept:

```
Buyer
  ↓
Payment
  ↓
Platform holds funds
  ↓
Artist delivers
  ↓
Completion / review
  ↓
Funds released
```

This is intended to create more trust than a normal direct transaction.

---

12. IMPORTANT: Prototype Scope

For the current prototype:

> **We are NOT implementing real payment movement yet.**

Reason:

The payment provider integration requires onboarding/verification and additional production requirements.

Therefore the prototype should simulate the workflow.

Example:

```
Buyer creates protected order
        ↓
Order marked "Payment Pending"
        ↓
Demo payment confirmation
        ↓
Order becomes "In Progress"
        ↓
Evidence submitted
        ↓
Admin reviews
```

```
        ↓
Admin marks completed
```

We can later replace the simulated payment state with a real payment provider.

Prototype goal

We are testing:

- UX
- workflow
- trust model
- admin operations
- database relationships
- product usability

We are NOT claiming the prototype contains a live financial escrow service.

---

13. Future Protected Transaction Workflow

Once payments are integrated:

```
User
 ↓
Create Order
 ↓
Payment
 ↓
Funds held according to provider/legal architecture
 ↓
Artist performs / ships
 ↓
Evidence
 ↓
Completion
 ↓
Release / refund / dispute
```

The exact legal and payment architecture must be researched before production launch.

---

14. Evidence System

For protected transactions, we want evidence that can help resolve disputes.

Examples:

Before Delivery

Artist can upload:

- Product-condition photos
- Packing video
- Preparation evidence
- Shipping proof

After Delivery

Buyer can upload:

- Received-item photos
- Unboxing video
- Condition evidence

For bookings:

- Event confirmation
- Completion evidence
- Relevant documentation

---

15. Admin Review

The original concept included review of the transaction conversation and submitted evidence.

For the prototype, the admin panel should allow authorized staff to inspect:

- Order details
- Buyer information
- Artist information
- Transaction timeline
- Related conversation
- Uploaded evidence
- Internal notes
- Current status

For production, privacy rules and access policies must be formally defined.

16. Chat Privacy Model

Normal conversations should remain private.

We do not want administrators casually reading everyone's conversations.

A future protected transaction can carry a notice explaining that relevant information may be accessed by authorized staff when necessary for:

- Dispute resolution
- Fraud prevention
- Verification
- Platform safety

The admin panel should expose only the information the reviewer is authorized to access.

---

17. Evidence Retention

The original idea was:

> Once the admin reviews the before/after videos and marks them as reviewed, the media should be deleted.

That is a good privacy goal, but the production system should use a formal retention policy.

Prototype:

```
Uploaded
 ↓
Admin reviews
 ↓
Marked reviewed
 ↓
Can be deleted
```

Production:

```
Uploaded
 ↓
Retention policy
 ↓
```

```
Review / dispute window
 ↓
Delete media
 ↓
Keep necessary metadata / audit record
```

We should not promise immediate deletion until the legal/operational retention policy is finalized.

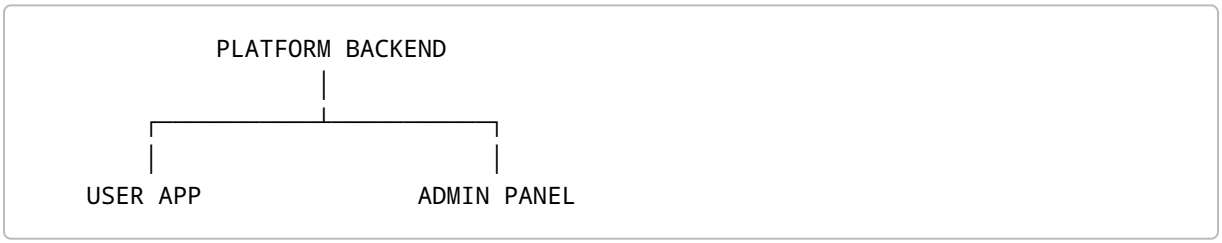
---

18. Admin Panel

The admin panel is a **separate web application**.

It is NOT part of the normal user app.

Think:



The normal user interacts with the public product.

Our internal team uses the admin panel.

Example:

```
admin.<our-domain>
```

During development it can simply run locally.

---

19. What Can the Admin Panel Do?

Dashboard

Show:

- Pending artist applications

- Active protected orders
- Open disputes
- Reports
- Recent activity
- Basic platform statistics

---

20. Artist Verification Queue

Admin sees:

```
Artist
Category
Portfolio
Social Profile
Verification Code
Submitted Date
Status
```

Actions:

```
View
Approve
Reject
Request Information
```

---

21. Order / Protected Deal Review

Admin sees:

```
Order ID
Artist
Buyer
Product / Booking
Amount (future)
Status
Created Date
```

Order details can include:

- Conversation
- Timeline
- Evidence
- Internal notes
- Status changes

Prototype actions:

```
Mark Reviewed
Request More Evidence
Mark Completed
Open Dispute
Refund Demo Transaction
```

Real payment release/refund comes later.

---

22. Disputes

Possible statuses:

```
OPEN
UNDER_REVIEW
WAITING_FOR_ARTIST
WAITING_FOR_BUYER
RESOLVED
CLOSED
```

Admin should be able to record:

- Decision
- Reason
- Evidence reviewed
- Internal notes
- Final outcome

---

23. Reports & Moderation

Users may report:

- Fake artist
- Fraud
- Spam
- Copyright concern
- Harassment
- Inappropriate content
- Suspicious transaction

Admin reviews and records the outcome.

---

24. Audit Logs

Every important admin action should be recorded.

Example:

```
Admin: user_102
Action: APPROVED_ARTIST
Artist: artist_43
Time: 2026-08-14 18:32
```

This becomes especially important once multiple employees are using the system.

---

25. Coming-Soon: Competitions

We want to introduce **artist competitions**.

Possible competition types:

- Singing
- Instrumental
- Painting
- Photography
- Digital Art
- Short Film
- Dance
- Poetry

- Design

Possible flow:

```
Competition announced
 ↓
Artists register
 ↓
Submission window
 ↓
Judging / community participation
 ↓
Results
 ↓
Winner recognition
```

Future features may include:

- Leaderboards
- Categories
- Judges
- Public voting
- Prizes
- Artist badges
- Featured winners

This should be treated as a **future product pillar**, not part of the first prototype unless a minimal version is needed.

---

26. Coming-Soon: Art Supplies

Another planned marketplace category is **art supplies**.

Potential products:

- Paint
- Brushes
- Canvas
- Sketchbooks
- Drawing tools
- Craft materials
- Digital-art accessories
- Music-related creative equipment later if strategically relevant

The long-term concept is:

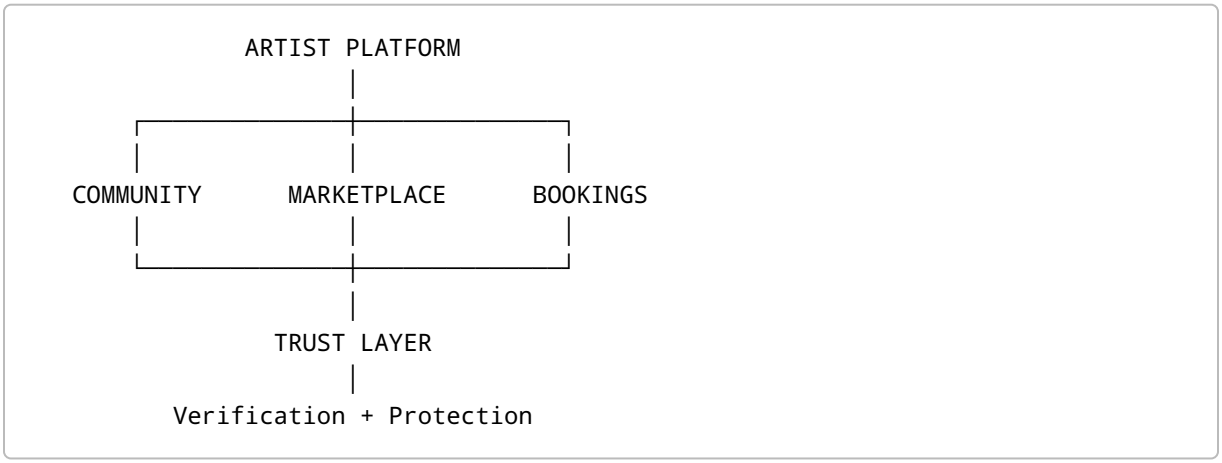
```
Discover Artist
      ↓
Become Artist
      ↓
Create
      ↓
Buy Supplies
      ↓
Sell Work
      ↓
Grow Audience
```

This can eventually become a major commerce category.

---

27. Long-Term Product Ecosystem

The platform can eventually have four major pillars:



Then future expansion:

```
Competitions
     +
Art Supplies
     +
AI Features
     +
Events
```

---

28. Initial MVP — What We Actually Build First

We should NOT build everything at once.

MVP Core

```
Authentication
Artist onboarding
Artist verification
User profiles
Artist profiles
Feed
Search
Follow/support
Posts
Basic chat
Product listings
Basic booking flow
Admin panel
Protected-deal simulation
Reports
Notifications
```

Deferred

```
Real payment integration
Real escrow settlement
Complex recommendation AI
Advanced competitions
Art-supply marketplace
Large-scale analytics
Advanced moderation AI
```

---

29. Recommended Technical Stack

Frontend — Web

React

Used for:

- Public web application
- Admin panel

We can potentially share components and design-system code between them where appropriate, while keeping separate applications/permissions.

---

30. Mobile

Flutter

One codebase can target:

- Android
- iOS

We should build mobile after the core product flows are validated.

---

31. Backend

Node.js + Express

Responsibilities:

- Authentication
- User management
- Artist verification
- Posts
- Chat
- Products
- Bookings
- Protected-order workflow
- Notifications
- Admin APIs
- Permissions

---

32. Database

PostgreSQL

Preferred because our system contains highly related transactional data:

```
Users
Artists
Orders
Bookings
Messages
Payments
Disputes
Verification
```

These relationships benefit from a relational database.

---

33. ORM

Prisma

Node.js ↔ PostgreSQL database interaction.

It gives us:

- Typed database access
- Migrations
- Schema management
- Easier developer workflow

---

34. Authentication

Initial options:

- Firebase Authentication
- Clerk

We will choose one after comparing:

- Cost
- Features
- Developer experience
- Scalability
- Control

---

35. File Storage

Do NOT put images/videos directly inside PostgreSQL.

Use object storage.

Possible options:

- Supabase Storage
- Cloudinary

Storage should handle:

- Profile photos
- Artist portfolios
- Product images
- Evidence
- Videos

---

36. Hosting

Possible initial structure:

```
React
 ↓
Vercel

Node.js API
 ↓
Render / equivalent

PostgreSQL
 ↓
Supabase

Media
 ↓
Cloudinary / Supabase Storage
```

We should confirm current free-tier limits before committing to a production architecture.

---

37. Caching & Performance

Caching is important.

Examples:

Images

First request:

```
Server → Image → Device cache
```

Later:

```
Cache → Image
```

Much faster.

Feed

Do not load 500 posts at once.

Use pagination:

```
Posts 1–10
↓
Posts 11–20
↓
Posts 21–30
```

Chat

Initially load the newest messages and fetch older messages when scrolling upward.

Videos

Use proper compression, thumbnails, CDN delivery and controlled preloading.

---

38. UX / Design Philosophy

The product should feel:

- Premium
- Smooth
- Modern
- Creative
- Trustworthy
- Fast
- Artist-focused

We are inspired by excellent product design, but we should not intentionally design manipulative addictive mechanics.

Instead, create rewarding moments around meaningful actions:

```
Artist verified
       ↓
Celebration

Artwork receives applause
       ↓
Positive feedback

Artist gets booking
       ↓
Success moment

Competition won
       ↓
Recognition
```

---

39. Mobile UX

Important principles:

- Thumb-friendly controls
- Large touch targets
- Bottom navigation where appropriate
- Important actions in easy-reach areas
- Minimal unnecessary typing
- Fast transitions

- Skeleton loading
- Clear empty states
- Good error states

---

40. Login / Onboarding Direction

The login design can follow the reference style the team has already discussed:

- Dark/premium visual direction
- Strong brand identity
- Minimal layout
- Clear primary CTA
- Google/social login where appropriate
- Email authentication
- Strong visual hierarchy

The exact colors, typography and spacing should be formalized into a design system before implementation.

---

41. Design System

We need a single source of truth for:

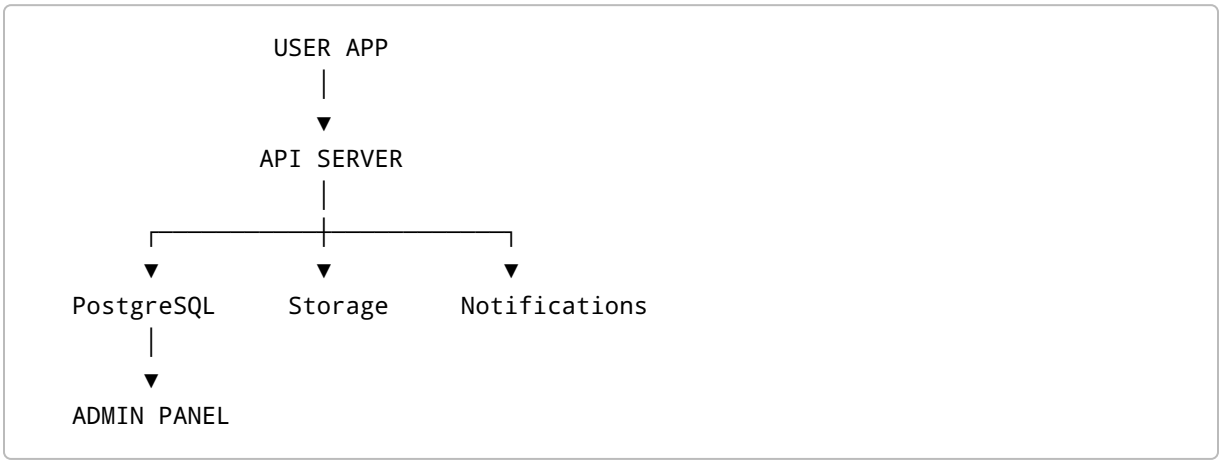
- Colors
- Typography
- Spacing
- Border radius
- Buttons
- Inputs
- Cards
- Bottom sheets
- Dialogs
- Icons
- Shadows
- Motion
- Accessibility states

Do not let every developer invent their own UI.

---

42. Core Development Architecture

High-level:



---

43. Example: Artist Verification

User App

```
Artist submits application
```

↓

Backend

```
POST /artist/applications
```

↓

Database

```
status = PENDING
```

↓

Admin Panel

Application appears in queue.

↓

Admin approves.

↓

Backend

```
status = VERIFIED
```

↓

User App

Artist sees:

```
Verified Artist ✓
```

---

44. Example: Protected Deal Prototype

```
User
 ↓
Creates order
 ↓
Backend creates order
 ↓
Payment status = DEMO_CONFIRMED
 ↓
Order status = IN_PROGRESS
 ↓
Artist submits evidence
 ↓
Buyer submits evidence
 ↓
Admin reviews
 ↓
Admin marks complete
```

Later:

```
DEMO_CONFIRMED
```

is replaced by the real payment provider's payment-confirmation flow.

---

45. Suggested Initial Repository Structure

At a high level:

```
artist-platform/
│
├── web/
│   ├── public/
│   ├── src/
│   └── ...
│
├── admin/
│   ├── src/
│   └── ...
│
├── mobile/
│   └── ...
│
├── backend/
│   ├── src/
│   │   ├── auth/
│   │   ├── users/
│   │   ├── artists/
│   │   ├── posts/
│   │   ├── chat/
│   │   ├── products/
│   │   ├── bookings/
│   │   ├── orders/
│   │   ├── disputes/
│   │   ├── notifications/
│   │   └── admin/
│   └── ...
│
└── docs/
```

The exact structure can change once the engineering team agrees on the architecture.

---

46. Development Philosophy

Do not build future complexity before users need it.

Priority:

```
Reliable core workflow
        ↓
Good UX
        ↓
Real user testing
        ↓
Feedback
        ↓
Iteration
        ↓
Scale
```

Not:

```
Build 100 features
        ↓
Launch
        ↓
Hope people like it
```

---

47. What the New Team Member Should Understand

The most important thing is this:

> We are not primarily building a social-media app.

We are building a **trusted artist ecosystem**.

The feed helps discovery.

The artist profile builds identity.

The marketplace creates commerce.

Bookings create services.

Verification creates authenticity.

Protected transactions create trust.

The admin system makes the whole operation possible.

Competitions create engagement and recognition.

Art supplies create another commerce layer.

---

48. Current Development Priority

For the current prototype, think in this order:

```
1. Authentication
2. User / Artist roles
3. Artist verification
4. Artist profile
5. Feed
6. Search / discovery
7. Chat
8. Product / booking basics
9. Protected-deal simulation
10. Admin panel
11. Moderation / reports
12. Notifications
```

Real payments come later.

Competitions come later.

Art supplies come later.

Advanced AI comes later.

---

49. What We Are Trying to Prove With the Prototype

The prototype should answer five questions:

1. Will artists join?

2. Do users actually want to discover artists here?

3. Will users buy or book through the platform?

4. Does the verification/trust concept make users feel safer?

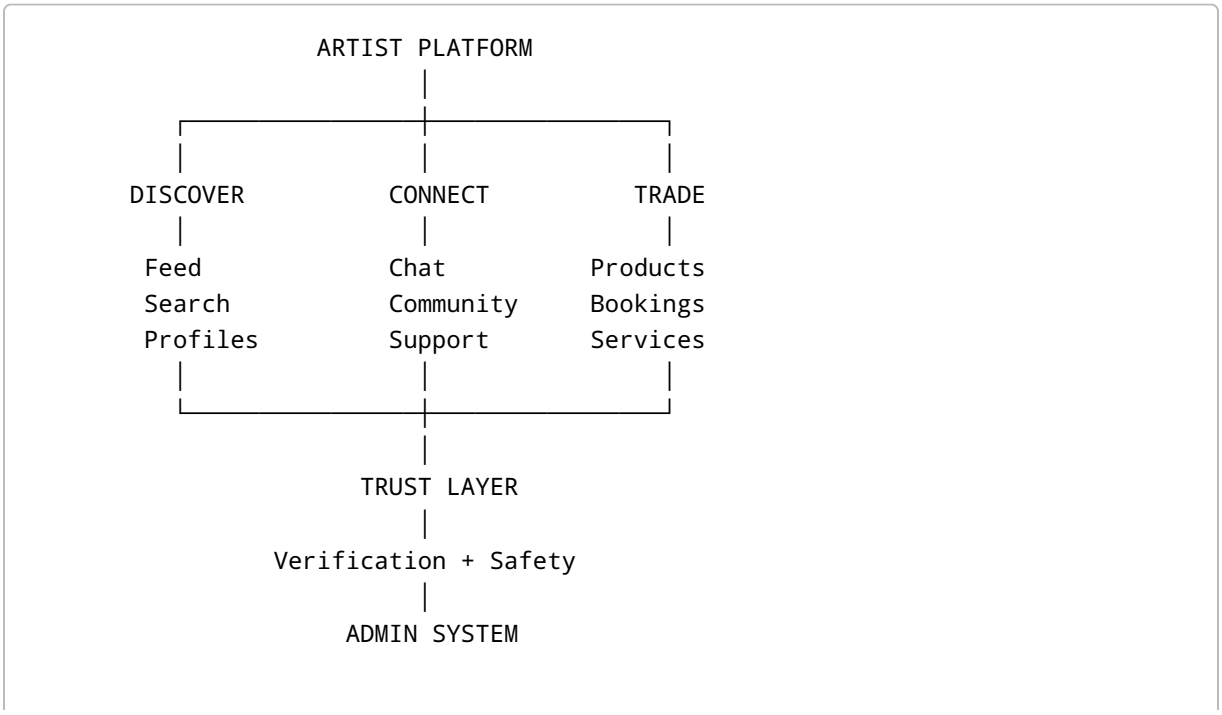
5. Can our team efficiently operate verification, disputes and moderation through the admin panel?

If the answer to these questions is positive, then deeper payment and marketplace infrastructure becomes worth investing in.

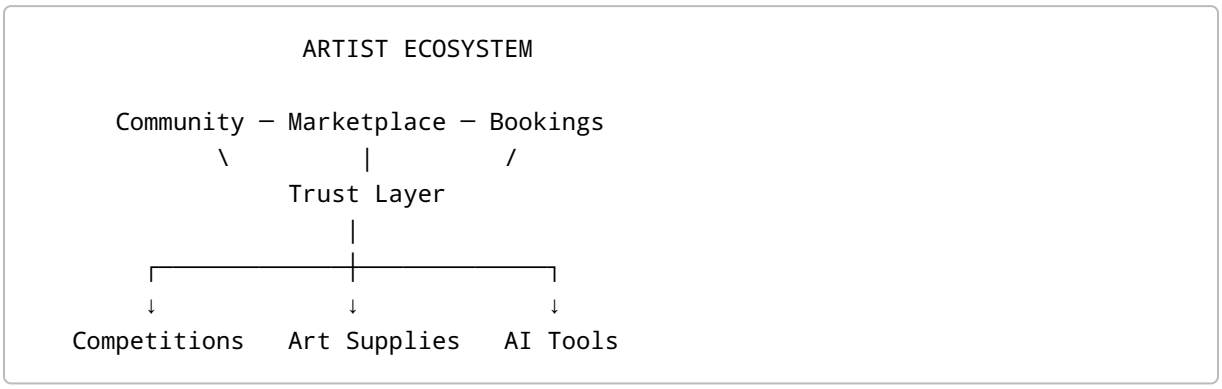
---

50. Final Mental Model for the Team

Think of the platform like this:



And the long-term expansion becomes:



---

51. The Rule for Everyone Joining the Team

Every feature should answer:

What user problem does this solve?

Why does it belong on our platform?

How does it improve the artist/user experience?

How does it affect trust, safety, or business?

If we cannot answer those questions, we should not build the feature just because it looks cool.

---

52. Current Status

✅ Concept

Defined.

✅ Core audience

Defined.

✅ Artist verification concept

Defined.

✅ Marketplace concept

Defined.

✅ Booking concept

Defined.

✅ Protected transaction concept

Defined.

✅ Admin portal concept

Defined.

✅ Technology direction

Defined.

🟡 Real payment integration

Deferred.

🟡 Competition system

Coming Soon.

🟡 Art-supply marketplace

Coming Soon.

🟡 AI systems

Future.

🟡 Production legal/payment architecture

Requires dedicated research before launch.

---

53. Immediate Next Step

Before coding aggressively, the team should convert this document into:

1. **MVP feature list**
2. **Screen-by-screen UX map**
3. **Database schema**
4. **API specification**
5. **Admin workflow specification**
6. **Design system**
7. **Git/GitHub workflow**
8. **Development task board**

That is the bridge between the **idea** and the **actual software**.